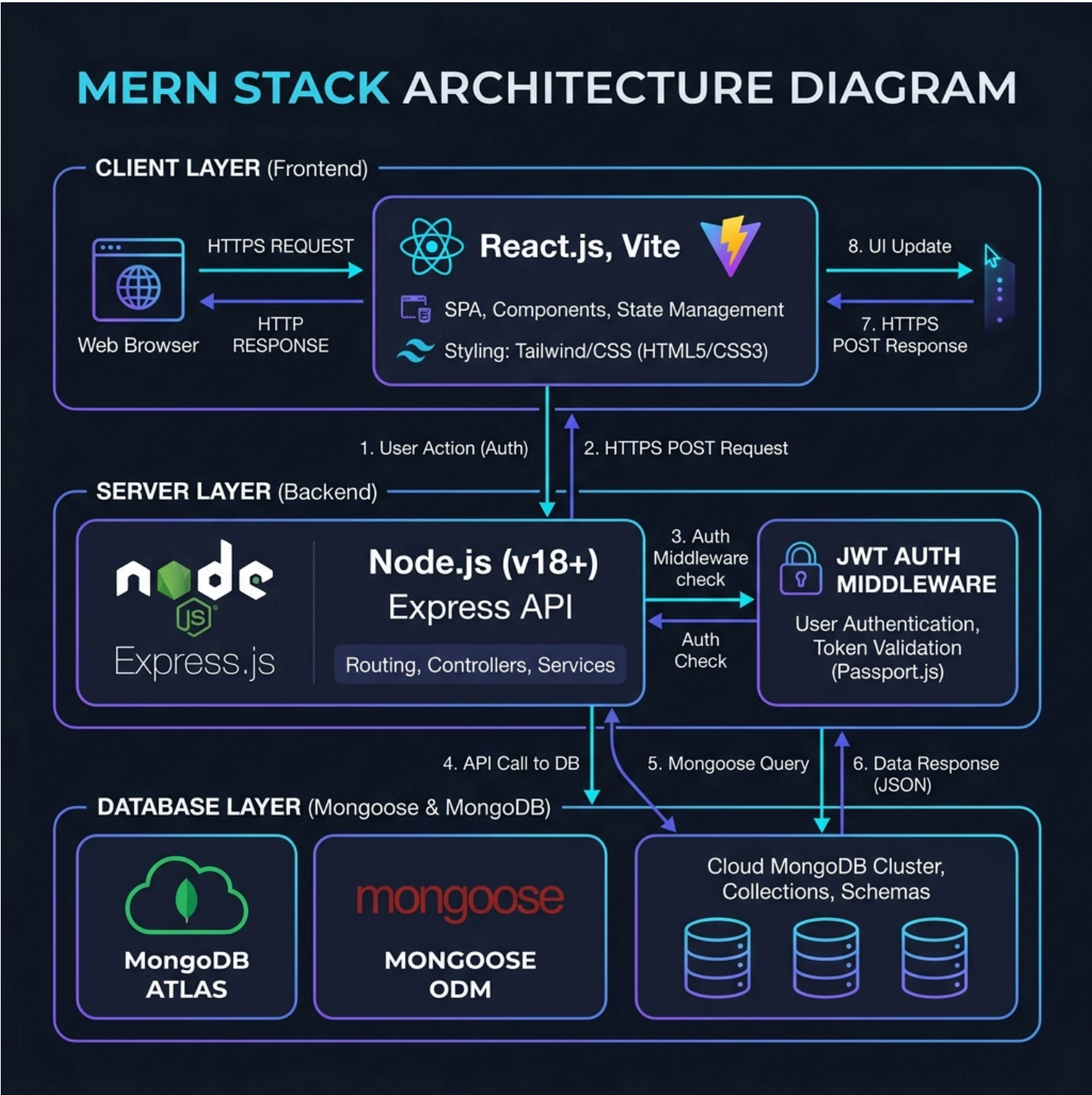


System Architecture - Student Performance Analytics Portal

This document outlines the high-level system architecture and request flow for the Student Performance Analytics Portal, structured using the MERN stack.

Technical Architecture Overview

The system follows a classic 3-tier client-server architecture model:



System Flowchart Blueprint

```
graph TD
    subgraph Client_Tier [Client Tier (Frontend)]
        A["React App (Vite SPA)"] --> B["Global State / Context"]
        A --> C["UI Component Views"]
    end

    subgraph Application_Tier [Application Tier (Backend)]
        D["Express.js Server"] --> E["Auth Middleware (JWT Validation)"]
        E --> F["Express Router Routing"]
        F --> G["Controller Functions"]
    end

    subgraph Database_Tier [Database Tier (Data Storage)]
        H["Mongoose ODM"] --> I["MongoDB Atlas Cluster"]
    end

    C -- "HTTP Requests (fetch / Axios)" --> D
    G --> H
    H --> I
    I --> H
    H --> G
    G -- "JSON Responses" --> C
```

Tier Explanations

1. Client Tier:
- Built with **React** bundled using **Vite**.
 - Responsible for rendering interactive components, graphing dashboard statistics, and managing user interaction states locally.
 - Securely stores JSON Web Tokens (JWT) in cookies/localStorage to include in subsequent HTTP requests.
2. Application Tier:
- Powered by **Node.js** running an **Express.js** API server.
 - Standard CORS and body-parser middleware process incoming requests.
 - Custom `authMiddleware` intercept secure endpoints, validating JWT signatures before resolving requests.
 - Routers forward requests to controllers, which encapsulate core business logic.
3. Database Tier:
- A document-oriented cloud-based database using **MongoDB** (accessed via **Mongoose**).
 - Designed for high-write loads and structured data models utilizing compound indices on analytic lookups.